

This is a great walkthrough of the full lifecycle from project creation to GitHub deployment. I've cleaned up the grammar, organized the flow, and used standard technical terms like "Staged Changes" and "Personal Access Token."

Experimental Procedure: Creating and Deploying a Maven Web App

Phase 1: Project Creation in Eclipse

1. **Launch Eclipse IDE.**
2. Navigate to **File > New > Maven Project.**
3. In the wizard, ensure "Create a simple project" is unchecked and click **Next.**
4. Set the **Catalog** to **Maven Central** and type `org.apache.maven.archetypes` in the filter.
5. Select **maven-archetype-webapp (1.5)** and click **Next.**
6. Enter the following coordinates:
 - o **Group Id:** `exp2`
 - o **Artifact Id:** `exp2`
7. Select **war** as the packaging type and click **Finish.**
8. When prompted in the console, type **Y** and press **Enter.** Ensure the console displays **BUILD SUCCESS.**

Phase 2: Configuration & Local Git Setup

9. In the **Project Explorer**, open `pom.xml`. Replace its contents with the updated code from your GitHub source and **Save.**
10. Right-click the project folder and select **Team > Share Project....**
11. Check the box **"Use or create repository in parent folder of project"**, click **Create Repository**, then click **Finish.**

Phase 3: GitHub Repository & Authentication

12. Log in to your GitHub account and create a **new repository.**
13. Copy the **Repository URL** from the "Quick Setup" page.
14. In GitHub **Settings > Developer Settings > Personal Access Tokens > Tokens (classic)**, generate a new token.
 - o **Note:** `exp2`
 - o **Scopes:** Check the `repo` box.
 - o **Action:** Copy the generated token (this will be your password).

Phase 4: Committing and Pushing to GitHub

15. In Eclipse, right-click the project and select **Team > Commit.**
16. Move all files from **Unstaged Changes** to **Staged Changes.**
17. Enter `First Commit` as the commit message and click **Commit.**
18. Right-click the project and select **Team > Push Head....**
19. In the **Remote URL** field, paste your GitHub repository link.
20. For authentication, enter your **GitHub Username** and paste your **Personal Access Token** as the password.

21. Click **Preview**, then **Push**.

Phase 5: Verification

22. Once the dialog box confirms "Pushed to GitHub," refresh your GitHub repository page to verify the files are present.

Conclusion: The build and deployment were successful. Experiment completed.

A Quick Tip for Next Time

Be very careful with your **Access Token**. Once you close that window on GitHub, you can't see the code again, so it's a good idea to keep it in a secure password manager!

Would you like me to help you format this into a formal Lab Report PDF or Markdown file?